

Discussion Unit:

DSE Analytics Scala
REPL statements
migrated to program

Migrate a set of DSE Analytics REPL statements, to a (true) Scala program

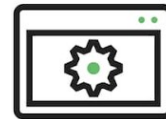
- Command line first, then IDE;
(Discussion Unit/Exercise 7556/7557)
- Software versions
- Install Scala, Maven
- Generate a pom.xml, change
- Generate an App.scala, change
- Submit as Spark job
- This Discussion Unit is dependent on Discussion Unit 7544/7545.

Discussion Lab:

Matching pairs – Match the attributes on the right with the areas on the left



**DSE
Java Driver**



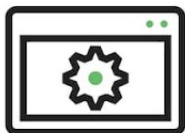
Spark
**DSE
Spark Cassandra
Connector**

Discussion Lab: Attributes

REPL



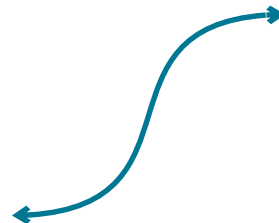
(True)
Program



Command
Line



IDE



Faster to develop



Deploy to production



More memory (laptop ?)



Less memory (laptop ?)



Client uses different IDE



Access to client network



DSE Spark client



DSE Java client

End of Discussion Lab:

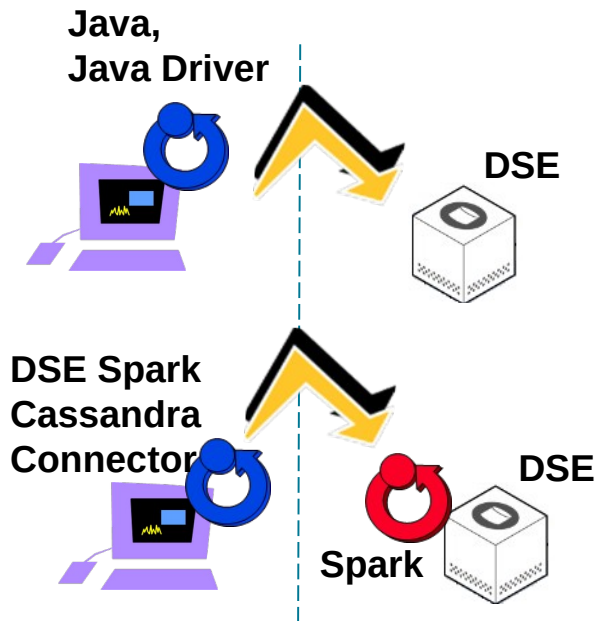
There are JDBC, ODBC, Parquet, (other) connectors built into Spark, but there is not a (Cassandra) one

DataStax provides 2 drivers that work interactively

- DSE Java Driver (See Discussion Unit, 6244/6245) (not Spark)
- Spark Cassandra Connector (Spark)
 - Open source, AND enterprise license features
 - Spark objects, activities

These allow you to connect to DSE directly from Spark, like using a JDBC driver, more

[What] is interacting with DSE ?



Versions: They Matter

DSE 6.0, 6.0.1
Java 8
Spark 2.2.1.2
Scala 2.11, 2.12

Connector	Spark	Cassandra	Cassandra Java Driver	Minimum Java Version	Supported Scala Versions
2.3	2.3	2.1.5*, 2.2, 3.0	3.0	8	2.11
2.0	2.0, 2.1, 2.2	2.1.5*, 2.2, 3.0	3.0	8	2.10, 2.11
1.6	1.6	2.1.5*, 2.2, 3.0	3.0	7	2.10, 2.11
1.5	1.5, 1.6	2.1.5*, 2.2, 3.0	3.0	7	2.10, 2.11
1.4	1.4	2.1.5*	2.1	7	2.10, 2.11
1.3	1.3	2.1.5*	2.1	7	2.10, 2.11
1.2	1.2	2.1, 2.0	2.1	7	2.10, 2.11
1.1	1.1, 1.0	2.1, 2.0	2.1	7	2.10, 2.11
1.0	1.0, 0.9	2.0	2.0	7	2.10, 2.11



Installing Scala:

Can already run Spark Scala via REPL,
dse spark

Or, installing Scala,
Check for Java JDK 1.8
javac -version

Then,
wget <http://downloads.typesafe.com/scala/2.11.8/scala-2.11.8.rpm>
yum install *.rpm



Apache Maven: What is it ?

Apache Maven is "build automation software". Think yum(C), or apt-get(C), (iTunes), but for Java programs.

- Will create your project filesystem structure, compile, deploy, other.
- Will find, download, version, and manage all of your Java/Scala dependencies.
- And, do all of this in about 4 total commands.
- Supports Java, Scala, Ruby, C/C++ others



Apache Maven: How to install



Run all commands in this unit inside a Linux terminal window.

- Already installed ? `"mvn --version"`
- If not, Tar ball; unzip, place in PATH
 - <https://maven.apache.org/download.cgi>
 - <https://maven.apache.org/install.html>
 - Version 3.5.4, 8MB in size
 - Is a Java app, requires Java 8 (v 3.5.4)
 - `mvn --version`

Previously (6244/6245) when using Java-

- `cd /opt` (cd anywhere)
- `mvn archetype:generate \`
 - DgroupId=`com.datastax.enablement.bootcamp` \
 - DartifactId=`my-app` \
 - Dversion=1.0 \
 - DinteractiveMode=false \
 - DarchetypeArtifactId=`maven-archetype-quickstart`

**Project name
(logical only)**



**Will be the Java
Package name (as
generated)**



Specific to Java



When using Scala-

- `cd /opt` (cd anywhere)
- `mvn archetype:generate \`
 - `-DgroupId=com.datastax.enablement.bootcamp \`
 - `-DartifactId=my-app \`
 - `-Dversion=1.0 \`
 - `-DinteractiveMode=false \`
 - `-DarchetypeArtifactId=scala-archetype-simple \`
 - `-DarchetypeGroupId=org.scala-tools.archetypes \`
 - `-DremoteRepositories=http://scala-tools.org/repo-releases`
- `cd my-app`
`rm -r ./src/test/scala/samples/*`



Different, calling for a Scala project



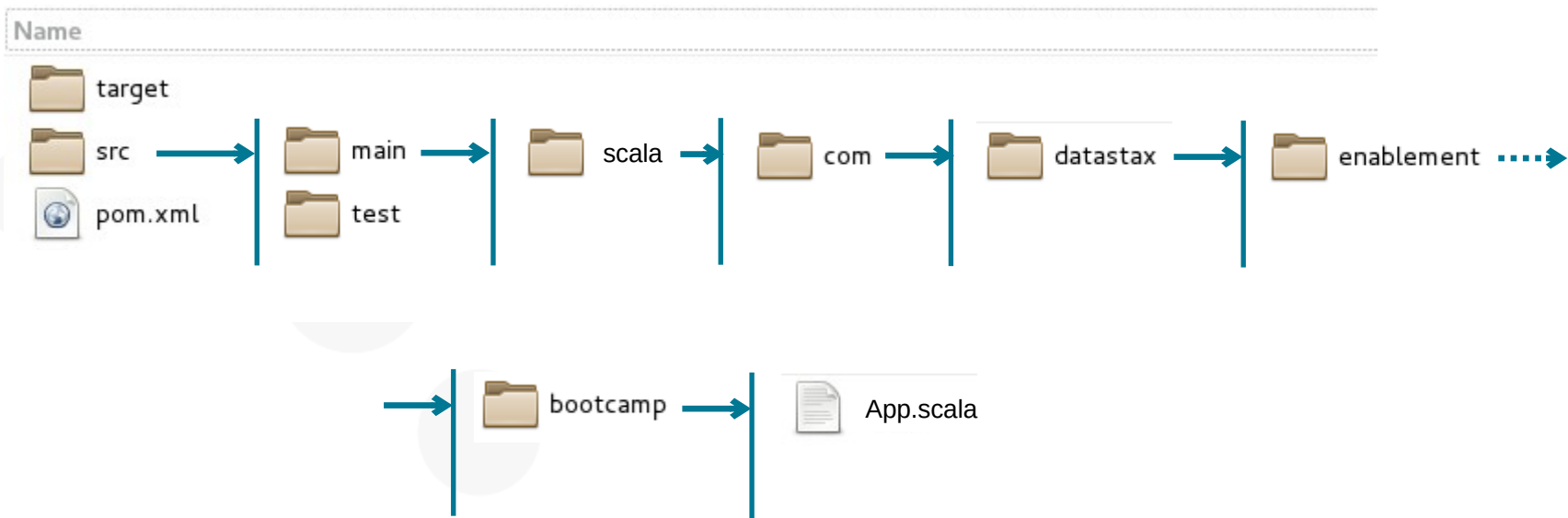
Specific to Scala



Generated tests: Will error on compile unless corrected

mvn generate: What just happened ?

-DgroupId=[com.datastax.enablement.bootcamp](#)



What is a pom.xml ?

- xml
- "project object model"
- Each project has one
- Tells Maven how to compile, etc, your project
- Edit once and done ?

```
<project
xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.datastax.enablement.bootcamp</groupId>
    <artifactId>my-app</artifactId>
    <version>1.0</version>
    <name>${project.artifactId}</name>
    <description>My wonderful scala

    ...    (Lines deleted)
```

What's in App.scala ?

```
package com.datastax.enablement.bootcamp

/**
 * @author ${user.name}
 */
object App {

  def foo(x : Array[String]) =
    x.foldLeft("")((a,b) => a + b)

  def main(args : Array[String]) {
    println( "Hello World!" )
    println("concat arguments = " + foo(args))
  }

}
```

- Stub
- It runs (it's Hello World)
- Add DSE client code; structure

Did Maven know about DSE/Spark ?

- No. (And versions.)
- Get a correct pom from DataStax
- Url on Notes page

java	6.0.1 support
scala	6.0.1 support
.gitignore	move to hivecontext and fix oss builds
BuildTestAll.sh	Turn off sbt output colors
LICENSE	Add license file
README.md	5.1.8 support

README.md

Example projects for using DSE Analytics

These are template projects that illustrate how to build Spark Application written in Gradle which can be run on either DataStax Enterprise (DSE) or Apache Spark. The simple write-to-/read-from-Cassandra application for each language and build tool

To compile and run a Java (Scala, kind of) program-

- From the "project directory"

```
cd /opt/my-app
```

```
-DgroupId=com.datastax.enablement.bootcamp
```

```
-DartifactId=my-app
```

- # compiles

```
mvn package
```

- # generates the Java CLASSPATH

```
mvn dependency:build-classpath -Dmdep.outputFile=cp.txt
```

- # call to run the program (one line below)

```
java -cp target/my-app-1.0.jar:`cat cp.txt`  
com.datastax.enablement.bootcamp.App
```

Left single
apostrophes



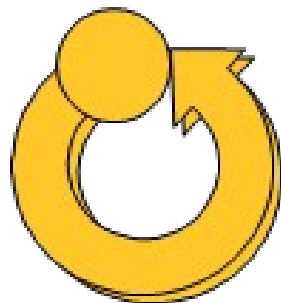
The better sequence for DSE/Spark/Scala: Next steps

- Edit, compile, test, (repeat)

From the parent directory of the project; from (./my-app/)

```
mvn package
```

```
dse spark-submit --class \
  com.datastax.enablement.bootcamp.App target/my-app-1.0.jar
```



New/odd Jars ? Put them in the pom.xml

DSE Spark Scala:

Example, end to end

- For brevity, this example does not display the two additional/derived columns.
- End to end example; imports, DSE Analytics (Spark) Session object, other

```
// Run this CQLSH block first, before running this program.
//
// DROP KEYSPACE IF EXISTS ks_7545;
// CREATE KEYSPACE ks_7545 WITH REPLICATION =
//     {'class': 'SimpleStrategy',
//      'replication_factor': 1};
// USE ks_7545;
//
// CREATE TABLE customer_plus_derived
// (
//     customer_num          INT PRIMARY KEY,
//     fname                 TEXT,
//     ... lines deleted
//     phone                 TEXT,
//     company_upshift       TEXT,    // This column not in CSV input file
//     ew_flag               TEXT    // This column not in CSV input file
// );
```

Just customer:
end to end
example-

Just customer: end to end example-

```
package com.datastax.enablement.bootcamp

import com.datastax.spark.connector._
    //
import com.datastax.spark.connector.cql.CassandraConnector
    //
import org.apache.spark.sql.{SaveMode, SparkSession}
import org.apache.spark.sql.cassandra._
    //
import org.apache.spark.sql.types._
```

Just customer: end to end example-

```
object App {
```

```
// If this case class is not outside the main,  
// we can get a compiler error,  
case class My_Record (  
  customer_num    : Int,  
  fname           : String,  
  lname           : String,  
  company          : String,  
  address1         : String,  
  address2        : String,  
  city             : String,  
  state            : String,  
  zipcode          : String,  
  phone           : String  
)
```

Just customer: end to end example-

```
def main(args : Array[String]) {  
  
    val spark = SparkSession.builder  
        .appName("My App")  
        .getOrCreate()  
    val sc = spark.sparkContext  
  
    // You will need this line eventually  
    //  
    // import spark.implicits._  
  
    // -----
```

Just customer: end to end example-

```
// Reading from a DSE table
//
// val rows10 = spark.sparkContext.cassandraTable(
//     "system", "local").select("dse_version")

val rows10 = sc.cassandraTable("system", "local").
    select("dse_version")

rows10.collect().foreach(println)
println()

// -----
```

```
// Reading a CSV, but as a text file; List of Strings
//
val rows20 = sc.textFile("file:///opt/stores_db/customer.csv")
rows20.collect().foreach(println)
println()

// split returns an array
//
val rows21 = rows20.map ( line => line.split (","))

// map the array elements (by position) into our record
//
val rows22 = rows21.map ( p => My_Record ( p(0).toInt,
    p(1).toString, p(2).toString, p(3).toString,
    p(4).toString, p(5).toString, p(6).toString,
    p(7).toString, p(8).toString, p(9).toString ))
rows22.collect().foreach(println)
println()
```

Just
customer:
end to end
example-

Just customer: end to end example-

```
rows22.saveToCassandra("ks_7545", "customer",  
    SomeColumns( "customer_num"    , "fname",  
        "lname"        , "company"        ,  
        "address1"     , "address2"     ,  
        "city"         , "state"         ,  
        "zipcode"      , "phone"         ))
```

```
// -----
```

```
// The above is subject to error; positional array reading
//
// Reading directly into a StructType; can't use a case class
// with the read.schema below-
//
val My_Schema = StructType(Array(
  StructField("customer_num"    , IntType, true),
  StructField("fname"           , StringType, true),
  StructField("lname"           , StringType, true),
  StructField("company"         , StringType, true),
  StructField("address1"        , StringType, true),
  StructField("address2"        , StringType, true),
  StructField("city"            , StringType, true),
  StructField("state"           , StringType, true),
  StructField("zipcode"         , StringType, true),
  StructField("phone"           , StringType, true)
))
```

Just
customer:
end to end
example-

```
// Reading the CSV into a StructType actually returns a DataFrame
//
// . Reading a CSV requires a schema (which gets us into DataFrame
//   territory)
// . That is cheating at this point
//
// . Reading as a text file, and parsing this string with confidence
//   is a lot of code we want to avoid.
//
val rows30 = spark.read.schema(My_Schema).csv(
    "file:///opt/stores_db/customer.csv")

// Convert the DataFrame to RDD
//
val rows31 = rows30.rdd
rows31.collect().foreach(println)
println()
```

Just customer:
end to end
example-

Just customer: end to end example-

```
// ks_7545.customer must exist
//
rows31.saveToCassandra("ks_7545", "customer",
    SomeColumns( "customer_num" , "fname",
        "lname"           , "company"           ,
        "address1"        , "address2"          ,
        "city"            , "state"             ,
        "zipcode"         , "phone"             ))

// -----

// We need these lines in order to terminate the program
//
spark.stop()
sys.exit(0)

}
}
```



End of Unit:



Additional Detail:

DSE Java Driver-

On top of the open source driver sits the DSE Driver-

- Specific extensions for DSE
 - Authenticator implementations that use the authentication scheme negotiation in the server-side DseAuthenticator
 - Value classes for geospatial types, and type codecs that integrate them seamlessly
 - Continuous Paging
 - DSE graph integration
 - <https://github.com/datastax/java-dse-driver>
- Only allowed to use if you have a valid DSE License

DSE Spark (Cassandra) Connector-

- Exposes (Cassandra) tables as Spark RDDs, DataFrames, Datasets
- Read, write, and more
- Execute arbitrary CQL queries in your Spark applications
- Used with the Cassandra Java Driver
- Open Source Apache 2.0 license
- DataStax Github Repository, <https://github.com/datastax/spark-cassandra-connector>

Use cases:

- Streaming data into DSE
- Analyzing the data in place
- Migrating from bad data model